



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

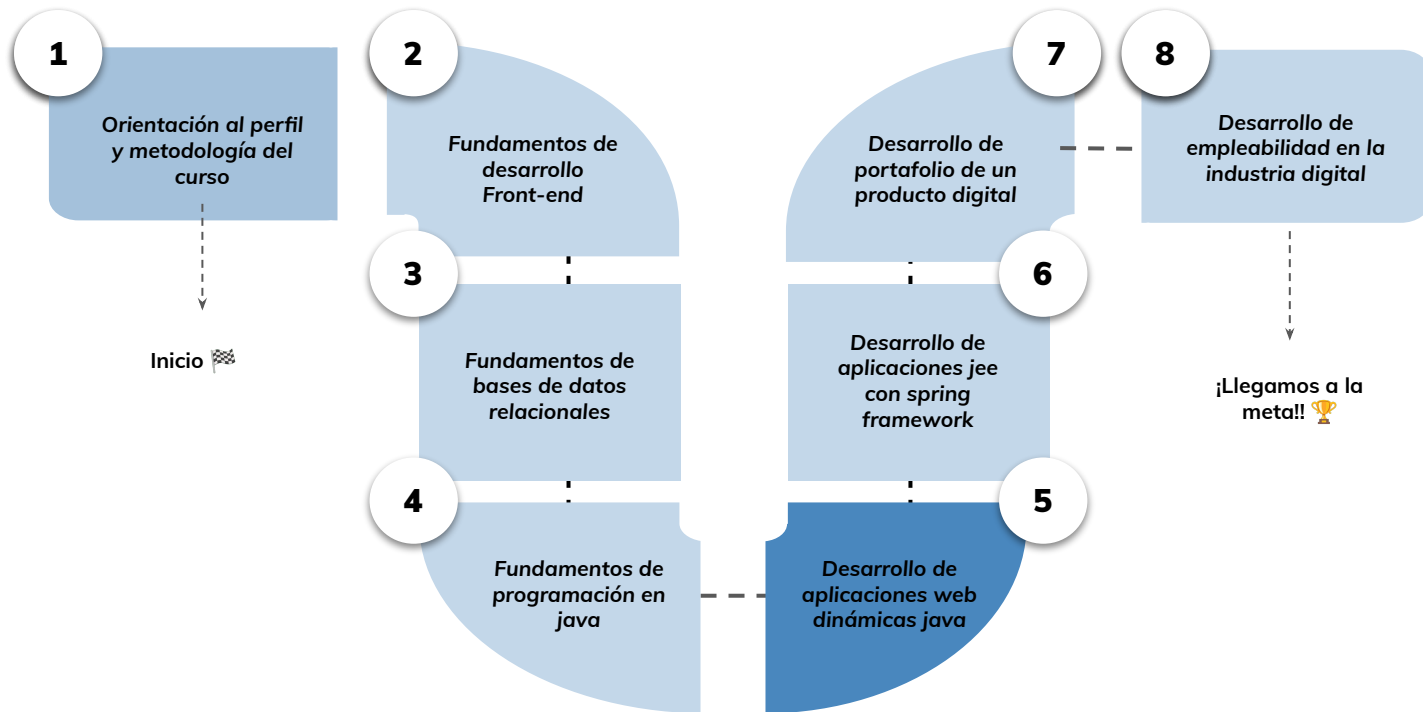


› Conociendo los servlets- Parte 2

Aprendizaje Esperado 3: Estructurar el comportamiento de una aplicación web dinámica utilizando servlets que procesan peticiones GET y POST para dar solución a un problema.

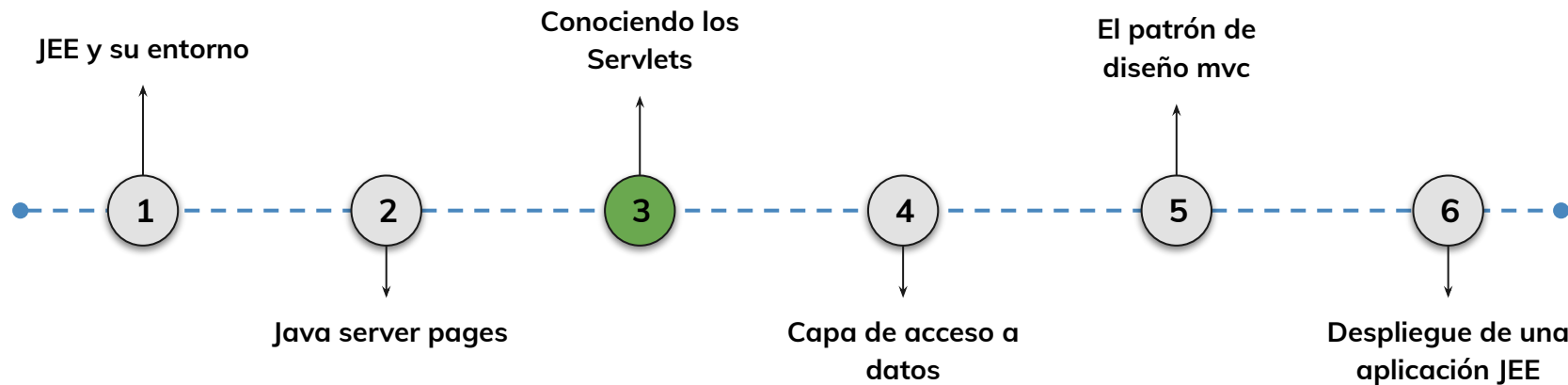
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

3.

Conociendo los Servlets

Repasaremos los parámetros de un POST request, paso de parámetros de un Servlet hacia una vista JSP y la implementación de un formulario con validación en el servlet

POST request

Controlando parámetros de un post request.

Envío de parámetros a JSP & Validación de formulario

Parametros a JSP

Validando en el servlet



Objetivos de aprendizaje

¿Qué aprenderás?



- Aprender a controlar parámetros de un POST request
- Aprender a pasar parámetros desde un Servlet a una vista JSP
- Comprender cómo implementar un formulario con validación en el servlet

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprender la configuración e implementación de los Servlets
- Conocer el manejo de sesiones y de cookies
- Comprender cómo compartir información entre servlets
- Conocer el concepto de concurrencia
- Comprender cómo controlar los parámetros de un GET request

➤ Controlando parámetros de un POST request

< > Controlando parámetros de un POST request

En el caso de recibir los parámetros desde un formulario, por ejemplo, se utiliza prácticamente la misma estructura que el método `doGet()`, pero en este caso se denomina `doPost()`.

Este método **solo está accesible desde los formularios**. Se envían los parámetros de forma **implícita** junto a la página, es decir, al pasar los parámetros, nosotros no vemos reflejado en ningún sitio qué parámetros son y cuál es su valor (están ocultos).



< > Controlando parámetros de un POST request

Veamos un ejemplo de cómo controlar los parámetros de una solicitud POST en un servlet:

Supongamos que tienes un servlet llamado MainServlet que maneja una solicitud POST con dos parámetros, "nombre" y "edad". El servlet mostrará estos parámetros en la respuesta al cliente.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    // Obtenemos el valor del parámetro "nombre" de la solicitud POST
    String nombre = request.getParameter("nombre");

    // Obtenemos el valor del parámetro "edad" de la solicitud POST
    String edad = request.getParameter("edad");

    // Procesamos y validamos los parámetros según sea necesario

    // Creamos la respuesta para el cliente
    response.setContentType("text/html;charset=UTF-8");
    response.getWriter().println("<html><body>");
    response.getWriter().println("<h1>¡Hola, " + nombre + "!</h1>");
    response.getWriter().println("<p>Tu edad es: " + edad + " años.</p>");
    response.getWriter().println("</body></html>");
}
```



< > Controlando parámetros de un POST request

Para verlo funcionando, vamos a modificar el servlet GET con el que habíamos trabajado antes para añadirle un formulario en donde solicitamos un parámetro al que llamaremos “nombre”.

La generación del formulario, la realizaremos en el método `doGet`, porque es donde vendrá, cuando llamemos a la página desde una URL.

Haremos que el formulario vuelva por POST, por lo que reservamos el método `doPost` para recibir los parámetros que el formulario envíe.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    // Obtenemos el valor del parámetro "nombre" de la solicitud POST
    String nombre = request.getParameter("nombre");

    // Obtenemos el valor del parámetro "edad" de la solicitud POST
    String edad = request.getParameter("edad");

    // Procesamos y validamos los parámetros según sea necesario

    // Creamos la respuesta para el cliente
    response.setContentType("text/html;charset=UTF-8");
    response.getWriter().println("<html><body>");
    response.getWriter().println("<h1>¡Hola, " + nombre + "!</h1>");
    response.getWriter().println("<p>Tu edad es: " + edad + " años.</p>");
    response.getWriter().println("</body></html>");
}
```



< > Controlando parámetros de un POST request

En la línea 39, enviamos el inicio del formulario, en donde indicamos que vamos a utilizar el método **POST**, ya que así, dejaremos el método GET para crear el formulario, y en **POST** lo recibiremos y procesaremos.

El indicar "ACTION=#" hace que el formulario se envíe a la misma página que lo creo, si no, deberíamos haber indicado la URL en donde se debía enviar.

Las restantes líneas crean un label, una caja de input (que tiene el nombre del parámetro que queremos recibir!!!) y un botón para enviar.

```
27-  
28  
29  
30  
31-  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
/**  
 * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse  
 *     response)  
 */  
protected void doGet(HttpServletRequest request, HttpServletResponse response)  
    throws ServletException, IOException {  
    response.setContentType("text/html");  
    PrintWriter out = response.getWriter();  
    out.println("<HTML>");  
    out.println("<HEAD><TITLE>Hola Mundo!</TITLE></HEAD>");  
    out.println("<BODY>");  
    out.println("Bienvenido a mi primera página Güev!");  
    out.println("<FORM METHOD=POST ACTION=#>");  
    out.println("<LABEL FOR=\"nombre\">Indique el nombre</LABEL>");  
    out.println("<INPUT NAME= \"nombre\"/>");  
    out.println("<INPUT TYPE= \"SUBMIT\"/>");  
    out.println("</FORM>");  
    out.println("</BODY></HTML>");  
}
```

< > Controlando parámetros de un POST request

Para recibir el parámetro 'nombre', llamaremos a `getParameter` en el método `doPost`, y luego indicamos que nos muestre la información en consola.

```
52- protected void doPost(HttpServletRequest request, HttpServletResponse response)
53-     throws ServletException, IOException {
54-     String nombre = request.getParameter("nombre");
55-     System.out.println(nombre);
56-
57- }
```



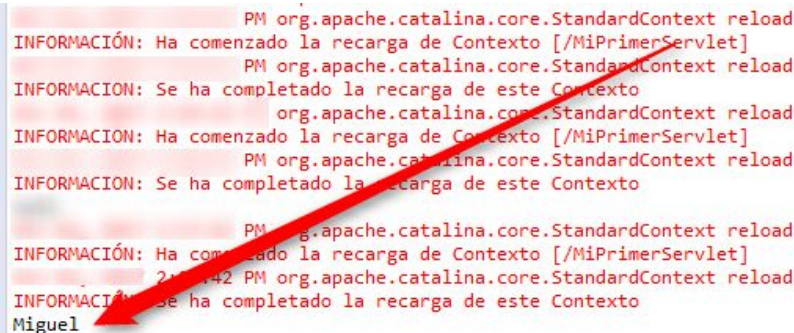
< > Controlando parámetros de un POST request

Si ahora intentamos lanzar el programa, veremos que en nuestro navegador aparece una caja de texto y un botón.



< > Controlando parámetros de un POST request

Escribimos nuestro nombre y pulsamos el botón “Enviar consulta” y en la consola de Eclipse veremos el nombre



The screenshot shows the Eclipse IDE console with several log messages. A red arrow points from the text 'Escribimos nuestro nombre...' to the word 'Miguel' in the console output. The messages are as follows:

```
PM org.apache.catalina.core.StandardContext reload
INFORMACIÓN: Ha comenzado la recarga de Contexto [/MiPrimerServlet]
PM org.apache.catalina.core.StandardContext reload
INFORMACIÓN: Se ha completado la recarga de este Contexto
PM org.apache.catalina.core.StandardContext reload
INFORMACIÓN: Ha comenzado la recarga de Contexto [/MiPrimerServlet]
PM org.apache.catalina.core.StandardContext reload
INFORMACIÓN: Se ha completado la recarga de este Contexto
PM org.apache.catalina.core.StandardContext reload
INFORMACIÓN: Ha comenzado la recarga de Contexto [/MiPrimerServlet]
2:07:42 PM org.apache.catalina.core.StandardContext reload
INFORMACIÓN: Se ha completado la recarga de este Contexto
Miguel
```



LIVE CODING

Ejemplo en vivo

Primer método doPost():

Vamos a crear un Servlet básico con los métodos vacíos para comenzar a implementar un método doPost:

- 1- Crear un servlet UserServlet con el método para procesar solicitudes GET que muestre un formulario para ingresar un nombre, un apellido y una contraseña.
- 2- Se debe validar que la contraseña sea '1234', si es correcta mostrar a través de un método doPost() los datos del usuario.

Si es incorrecta, mostrar un mensaje por consola que lo indique.

Tiempo: 30 minutos



Ejercicio N° 1

POST request

< > POST request

Manos a la obra: 🙌

En la clase 'UserServlet', implementar en el método doPost() en respuesta a los parámetros enviados por un formulario en doGet() .

Consigna: 📝

- 1- En el proyecto AlkeWallet, trabajar sobre el servlet llamado UserServlet
- 2- Crear un formulario de login en doGet() y responder un saludo personalizado en un método doPost() con el nombre de usuario.

Tiempo 🕒: 30 minutos



➤ Paso de parámetros de un Servlet hacia una vista JSP

< > Paso de parámetros de Servlet a JSP

Para pasar parámetros desde un servlet a una página JSP, puedes utilizar el objeto **request** o el objeto **session**. La elección entre uno u otro dependerá de si deseas que los parámetros sean visibles en la URL o si necesitas mantenerlos disponibles a lo largo de la sesión del usuario.

Cuando el cliente (navegador) solicita una URL, esta se envía al servidor. **En el servidor, se ejecutan archivos .jsp y .java** . Cuando, por ejemplo, un index.jsp llamar a un servlet(.java) estos se **comparten variables** , de forma que el resultado de una variable en el servlet, se le pueda mandar al jsp, y viceversa.



< > Paso de parámetros de Servlet a JSP

Page Scope

Son variables que solo se ven dentro de nuestro JSP o Servlet mientras se está ejecutando. Una vez construída y enviada la respuesta al navegador, desaparecen. Su única utilidad es ayudarnos a construir la página que queremos mostrar en el navegador.

```
<% int variable = 33; %>
...
<p>La variable vale <%= variable %></p>
```

y dentro de un Servlet no serían más que variables locales de nuestros métodos doGet() o doPost().

```
public class UnServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        int variable = 33;
        ...
        resp.getWriter().print("<p> La variable vale " + variable + "</p>");
    }
}
```



< > Paso de parámetros de Servlet a JSP

Request Scope

Estas variables **son más útiles** . Las guardamos en la **petición** que nuestro JSP/Servlet van a hacer a otro JSP/Servlet de nuestra aplicación, y este segundo JSP/Servlet puede recogerlos para hacer cosas con ellas.

Dentro de un JSP tenemos predefinido un objeto **request**, y en él podríamos fijar estas variables con **setParameter()** o con **setAttribute()**. y obtener los valores con los **getParameter()** o **getAttribute()**.



< > Paso de parámetros de Servlet a JSP

Puedes agregar atributos al objeto **request** en el servlet y acceder a estos atributos en la página JSP utilizando la expresión **\${}**.

```
public class MainServlet extends HttpServlet {  
  
    protected void doGet(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
        // Obtener el valor del parámetro "nombre" de algún lugar (base de datos, etc.)  
        String nombre = "Juan";  
  
        // Agregar el parámetro "nombre" al objeto request  
        request.setAttribute("nombre", nombre);  
  
        // Redireccionar a la página JSP  
        request.getRequestDispatcher("miPagina.jsp").forward(request, response);  
    }  
}
```

De esta manera en la página JSP puedes acceder al parámetro utilizando **\${nombre}**:

```
*miPagina.jsp X  
1  
2 <!DOCTYPE html>  
3 <html>  
4 <head>  
5     <title>Mi Página JSP</title>  
6 </head>  
7 <body>  
8     <h1>Hola, ${nombre}!</h1>  
9 </body>  
10 </html>
```

< > Paso de parámetros de Servlet a JSP

Con esta configuración, podrás acceder a la vista jsp directamente desde la url del servlet:



Hola, Juan!



< > Paso de parámetros de Servlet a JSP

Session Scope

Estas variables se mantienen durante lo que se conoce como una sesión. Cuando un usuario visita nuestra aplicación por primera vez, **automáticamente se crea una sesión para ese usuario**. Esta sesión **suele estar abierta mientras el usuario va navegando por las páginas** de nuestra aplicación y desaparece cuando el usuario deja de navegar por nuestra aplicación durante un tiempo predeterminado.

Dicho de otra forma, **cualquier valor que guardemos en la sesión del usuario, sólo será visible por las páginas que visite ese usuario y mientras el usuario esté activo**. Son las variables típicas donde guardar si un usuario ha entrado en sesión con un usuario y password, su nombre, su carrito de la compra si nuestra aplicación es de comercio electrónico, etc.



< > Paso de parámetros de Servlet a JSP

Si necesitas mantener los parámetros disponibles durante toda la sesión del usuario, puedes utilizar el objeto **session** para ello. Los atributos del objeto **session** **estarán disponibles hasta que la sesión del usuario expire** o se invalide.

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    // Obtener el valor del parámetro "nombre" de algún lugar (base de datos, etc.)
    String nombre = "Juan";

    // Obtener la sesión actual o crear una nueva
    HttpSession session = request.getSession(true);

    // Agregar el parámetro "nombre" al objeto session
    session.setAttribute("nombre", nombre);

    // Redireccionar a la página JSP
    response.sendRedirect("miPagina.jsp");
}
```



< > Paso de parámetros de Servlet a JSP

De esta manera en la página JSP (miPagina.jsp), puedes acceder al parámetro utilizando `${sessionScope.nombre}`:

```
miPagina.jsp x MainServlet.java
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Mi Página JSP</title>
5 </head>
6 <body>
7   <h1>Hola, ${sessionScope.nombre}!</h1>
8 </body>
9 </html>
10
```



< > Paso de parámetros de Servlet a JSP

En ambos casos (**request** y **session**), los parámetros pasados desde el servlet estarán disponibles en la página JSP y podrás mostrarlos o utilizarlos según tus necesidades.

Es importante recordar que **los atributos agregados al objeto request solo estarán disponibles durante la ejecución de la solicitud actual, mientras que los atributos agregados al objeto session estarán disponibles durante toda la sesión del usuario** .



LIVE CODING

Ejemplo en vivo

Compartiendo información:

Vamos a compartir información entre los servlets y las JSP:

- 1- Crear un formulario JSP llamado registro.jsp que pida al usuario un nombre, apellido, RUT y fecha de nacimiento (string)
- 2- Crear un Servlet que reciba esta información y devuelva un mensaje de bienvenida mostrando todos los datos del usuario en el navegador.

Tiempo: 20 minutos



Ejercicio N° 1

Parametros a JSP

< > Parametros a JSP

Manos a la obra: 🙌

Crear un archivo JSP en el proyecto AlkeWallet para registro de usuario.

Consigna: 📝

- 1- JSP llamado registro.jsp que pida al usuario un nombre, apellido, RUT, fecha de nacimiento (string) y contraseña.
- 2- Crear un Servlet que reciba esta información y devuelva un mensaje de bienvenida mostrando todos los datos del usuario en el navegador (excepto la contraseña).

Tiempo 🕒: 20 minutos



➤ Implementando un formulario con validación en el servlet

< > Formulario

Implementando un formulario con validación en el servlet

Como hemos visto, el método que utilizamos para recuperar los datos de los parámetros es `getParameter("X")`. Esta recuperación puede generar resultados diferentes:

- Cuando el campo "X" **no existe** , **obtenemos el valor `null`** .
- Cuando el campo "X" existe, pero **no tiene valor** , **obtenemos el valor vacío (`isEmpty`)** .
- Cuando el campo "X" **existe y tiene valor** , **obtenemos la cadena** de caracteres que corresponde a dicho valor.

Es habitual requerir que determinados campos tengan valor (campos requeridos), y en muchos casos es deseable que los valores de los campos cumplan determinadas propiedades



< > Formulario

Implementando un formulario con validación en el servlet

Para asegurarnos de que los datos que se ingresaron en el formulario son correctos o aceptables, vamos a realizar la validación en el servlet.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    // Obtener los parámetros enviados desde el formulario
    String nombre = request.getParameter("nombre");
    String email = request.getParameter("email");

    // Realizar la validación

    if (nombre == null || nombre.trim().isEmpty() || email == null || email.trim().isEmpty()) {
        // Si hay campos vacíos, redirigir al usuario de vuelta al formulario con un
        // mensaje de error
        response.sendRedirect("registro.html?error=1");
    } else {
        // Si la validación es exitosa, redirigir al usuario a una página de éxito
        response.sendRedirect("exito.html");
    }
}
```



< > Formulario

Implementando un formulario con validación en el servlet

En la vista JSP debemos tener en cuenta marcar con el atributo **required** los inputs cuyos datos sean requeridos para procesar.

```
<!DOCTYPE html>
<html>
<head>
  <title>Formulario de Registro</title>
</head>
<body>
  <h1>Registro de Usuario</h1>
  <form action="registroServlet" method="post">
    <label for="nombre">Nombre:</label>
    <input type="text" name="nombre" id="nombre" required>
    <br>
    <label for="email">Email:</label>
    <input type="email" name="email" id="email" required>
    <br>
    <input type="submit" value="Registrarse">
  </form>
</body>
</html>
```



LIVE CODING

Ejemplo en vivo

Validando datos en el servlet:

Vamos a validar todos los datos ingresados en el formulario creado en el ejercicio anterior :

- 1- Validar los datos enviados por registro.jsp.
- 2- Crear un mensaje de error que dispare el Servlet en caso de no pasar la validación.

Tiempo: 15 minutos



Ejercicio N° 1

Validando en el servlet

< > Parametros a JSP

Manos a la obra: 🙌

Vamos a validar todos los datos ingresados en el formulario creado en el ejercicio anterior.

Consigna: ✍️

- 1- Validar los datos enviados por registro.jsp.
- 2- Crear un mensaje de error que dispare el Servlet en caso de no pasar la validación.

Tiempo 🕒: 15 minutos





Momento:

Time-out!



5 -10 min.



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Comprender cómo controlar los parámetros de un POST request**
- ✓ **Comprender cómo enviar información de un Servlet a una JSP**
- ✓ **Aprender a realizar validaciones en el servlet**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son claves para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

